

Day 15 Part 2 — Multi-Class Classification & Iris Dataset

CrossEntropyLoss · Softmax · argmax · Real Data

Target: Google · Meta · OpenAI · Anthropic

The Big Picture

Days 13-15 Part 1 used binary classification — hired or not hired. Most real problems have more than 2 classes. Today you learn multi-class classification using the Iris flower dataset — 3 species to classify from 4 measurements. This introduces CrossEntropyLoss, Softmax, and argmax — tools you will use in every image classifier from Day 16 onwards.

File	What it does
day15_iris.py	Full multi-class pipeline on real Iris data with save/load

Concept 1: The Iris Dataset — Your First Real Data

The Iris dataset has 150 flower samples, 4 measurements each, 3 species to classify. Simple enough to train fast, real enough to be meaningful. Expected accuracy: 93-100% — Iris has well-separated classes.

Feature	What it measures	Example value
sepal_length	Length of the sepal (outer petal) in cm	5.1
sepal_width	Width of the sepal in cm	3.5
petal_length	Length of the inner petal in cm	1.4
petal_width	Width of the inner petal in cm	0.2
species (target)	0=Setosa, 1=Versicolor, 2=Virginica	0

```
from sklearn.datasets import load_iris

iris = load_iris()
X = iris.data      # shape (150, 4)
y = iris.target    # shape (150,) - integers 0, 1, 2

print(f"X shape: {X.shape}")
print(f"y shape: {y.shape}")
print(f"Classes: {iris.target_names}") # ['setosa' 'versicolor' 'virginica']
```

Concept 2: Binary vs Multi-Class — What Changes

Moving from 2 classes to 3 classes changes four things. Everything else — Dataset, DataLoader, training loop structure — stays the same.

	Binary (Days 13-15 Pt1)	Multi-class (Day 15 Pt2)
Output neurons	1	3 — one per class
Output activation	Sigmoid	None — CrossEntropyLoss does it
Loss function	BCELoss	CrossEntropyLoss
Label dtype	torch.float32 (0.0 or 1.0)	torch.long (integer: 0, 1, or 2)
Prediction	output > 0.5	torch.argmax(output, dim=1)

Concept 3: CrossEntropyLoss — Multi-Class Loss

CrossEntropyLoss is the standard loss for 3+ class classification. It combines two operations in one step: Softmax (converts raw scores to probabilities) and cross-entropy (measures how wrong the probabilities are). Do NOT apply sigmoid or softmax before CrossEntropyLoss — it does that internally.

What Softmax does:

```
Input logits (raw network output): [2.0, 1.0, 0.1]
After Softmax:                      [0.66, 0.24, 0.10] (sum to 1.0)
Predicted class:                    0 (highest probability)
```

How to construct CrossEntropyLoss:

```
import torch.nn as nn

criterion = nn.CrossEntropyLoss()

# output: raw logits from network — shape (batch_size, n_classes)
# target: integer class labels — shape (batch_size,) as torch.long
output = model(X_batch)          # shape (32, 3)
loss = criterion(output, y_batch) # y_batch must be Long integers

# IMPORTANT:
# Do NOT do: output = softmax(model(X_batch)) then criterion(output, y)
# CrossEntropyLoss applies softmax internally — doing it yourself is wrong
```

Label dtype matters — must be torch.long:

```
# WRONG — float labels cause an error with CrossEntropyLoss
y_train = torch.tensor(y_tr, dtype=torch.float32) # wrong

# CORRECT — long (integer) labels
y_train = torch.tensor(y_tr, dtype=torch.long)    # correct
```

Concept 4: torch.argmax — Making Predictions

In binary classification, prediction = output > 0.5. In multi-class, the network outputs 3 scores (one per class). The predicted class is the one with the highest score. torch.argmax finds that index.

```
with torch.no_grad():
    output = model(X_test)          # shape (30, 3) – 30 test samples, 3 class scores

    # argmax finds index of highest value per row
    # dim=1 means: find max across columns (across classes), one result per row
    predicted = torch.argmax(output, dim=1)  # shape (30,) – class index per sample

    # Example:
    # output[0] = [2.1, 0.3, -0.5] -> argmax = 0 -> predicted class: Setosa
    # output[1] = [0.1, 1.8, 0.4] -> argmax = 1 -> predicted class: Versicolor

    correct = (predicted == y_test).sum().item()
    accuracy = correct / len(y_test)
    print(f"Accuracy: {accuracy:.3f}")
```

Concept 5: Adam Optimizer — Better Than SGD

Adam (Adaptive Moment Estimation) adapts the learning rate for each parameter automatically. Parameters that get large gradients get smaller learning rates. Parameters with small gradients get larger learning rates. In practice Adam trains faster and more reliably than SGD — it is the default choice for most neural networks.

```
import torch.optim as optim

# SGD – simple, one learning rate for all parameters
optimizer = optim.SGD(model.parameters(), lr=0.01)

# Adam – adaptive, usually trains better and faster
# lr=0.001 is the standard starting point for Adam
optimizer = optim.Adam(model.parameters(), lr=0.001)

# Usage is identical – same 5-step training loop
optimizer.zero_grad()
loss.backward()
optimizer.step()
```

Full Code — day15_iris.py

```
import torch
import torch.nn as nn
import torch.optim as optim
import numpy as np
from torch.utils.data import Dataset, DataLoader
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler

# DATASET
class IrisDataset(Dataset):
    def __init__(self, X: torch.Tensor, y: torch.Tensor) -> None:
        self.X = X
        self.y = y
    def __len__(self) -> int:
        return int(self.X.shape[0])
    def __getitem__(self, idx: int) -> tuple[torch.Tensor, torch.Tensor]:
        return self.X[idx], self.y[idx]

# LOAD REAL DATA
iris = load_iris()
X_np = iris.data.astype(np.float32)
y_np = iris.target # integers 0, 1, 2

X_tr, X_te, y_tr, y_te = train_test_split(X_np, y_np, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_tr = scaler.fit_transform(X_tr)
X_te = scaler.transform(X_te)

# Float32 for features, Long (integer) for labels – required by CrossEntropyLoss
X_train = torch.tensor(X_tr, dtype=torch.float32)
X_test = torch.tensor(X_te, dtype=torch.float32)
y_train = torch.tensor(y_tr, dtype=torch.long)
y_test = torch.tensor(y_te, dtype=torch.long)

# DATALOADERS
train_loader = DataLoader(IrisDataset(X_train, y_train), batch_size=16, shuffle=True)
test_loader = DataLoader(IrisDataset(X_test, y_test), batch_size=16, shuffle=False)
print(f"Training samples: {len(X_train)}, Test samples: {len(X_test)}")
print(f"Training batches: {len(train_loader)}")

# NETWORK – 3 output neurons, NO sigmoid on output
class IrisNet(nn.Module):
    def __init__(self) -> None:
        super().__init__()
        self.layer1 = nn.Linear(4, 16)
        self.layer2 = nn.Linear(16, 8)
        self.layer3 = nn.Linear(8, 3) # 3 outputs – one per class
        self.relu = nn.ReLU()
        # NO sigmoid – CrossEntropyLoss handles it internally

    def forward(self, x: torch.Tensor) -> torch.Tensor:
        x = self.relu(self.layer1(x))
        x = self.relu(self.layer2(x))
        x = self.layer3(x) # raw logits – no activation
        return x

# SETUP
```

```

torch.manual_seed(42)
model = IrisNet()
criterion = nn.CrossEntropyLoss() # handles softmax internally
optimizer = optim.Adam(model.parameters(), lr=0.001)
print(f"Parameters: {sum(p.numel() for p in model.parameters())}")

# TRAINING LOOP
print("\nTraining on Iris...")
model.train()
for epoch in range(100):
    epoch_loss = 0.0
    for bX, by in train_loader:
        optimizer.zero_grad()
        output = model(bX) # shape (batch, 3)
        loss = criterion(output, by) # by must be Long
        loss.backward()
        optimizer.step()
        epoch_loss += loss.item()
    if (epoch + 1) % 10 == 0:
        avg = epoch_loss / len(train_loader)
        print(f" Epoch {epoch+1:3d} | Avg Loss: {avg:.4f}")

# EVALUATE
model.eval()
correct = 0
with torch.no_grad():
    for bX, by in test_loader:
        predicted = torch.argmax(model(bX), dim=1)
        correct += (predicted == by).sum().item()
accuracy = correct / len(y_test)
print(f"\nTest accuracy: {accuracy:.3f} ({correct}/{len(y_test)})")

# SAVE
torch.save(model.state_dict(), 'iris_model.pth')
print("Model saved to iris_model.pth")

# LOAD AND VERIFY
loaded = IrisNet()
loaded.load_state_dict(torch.load('iris_model.pth', map_location='cpu'))
loaded.eval()
with torch.no_grad():
    c2 = sum((torch.argmax(loaded(bX), dim=1) == by).sum().item() for bX, by in test_loader)
print(f"Loaded accuracy: {c2/len(y_test):.3f}")
print(f"Matches: {abs(accuracy - c2/len(y_test)) < 0.001}")

```

Expected Results

Metric	Expected	Notes
Training samples	120	80% of 150
Test samples	30	20% of 150
Loss at epoch 10	~0.5-0.9	Still learning early phases
Loss at epoch 100	~0.05-0.20	Well converged
Test accuracy	~93-100%	Iris is easy — high accuracy expected

Loaded matches	True	Save/load must be verified
----------------	------	----------------------------

Anki Cards — Day 15 Part 2

Q: What does CrossEntropyLoss do internally?

A: Applies Softmax then computes cross-entropy — do NOT apply softmax yourself

Q: What dtype must labels be for CrossEntropyLoss?

A: torch.long (64-bit integer) — not float

Q: What does torch.argmax(output, dim=1) return?

A: Index of highest value per row — the predicted class

Q: How many output neurons for 3-class classification?

A: 3 — one per class, with no activation on output layer

Q: What is Softmax?

A: Converts raw scores (logits) to probabilities that sum to 1.0

Q: What is Adam optimizer?

A: Adaptive optimizer that adjusts learning rate per parameter — better than SGD usually

Q: What is lr=0.001 for Adam?

A: Standard starting learning rate for Adam optimizer

Q: Binary vs multi-class: what changes?

A: Output neurons (1 vs N), loss (BCE vs CrossEntropy), labels (float vs long), prediction (>0.5 vs argmax)

Q: Why no sigmoid before CrossEntropyLoss?

A: CrossEntropyLoss applies softmax internally — adding sigmoid yourself gives wrong gradients

Q: What is the Iris dataset?

A: 150 flower samples, 4 measurements, 3 species — classic multi-class benchmark

Next: Day 16 — Convolutional Neural Networks (CNNs). Image classification begins.